

UFMA – Universidade Federal do Maranhão

Integrantes do Grupo: Millena Gomes Andrade de Menezes Braga, Cássio Herberth Rodrigues Araújo, Stefany Costa de Almeida, Arthur Felipe Mourão de Oliveira, Alana Mayara Silva Monteiro

Professor Orientador: Luiz Henrique Neves Rodrigues

Desenvolvimento do Jogo “Snake” em Linguagem Assembly (MIPS)

Relatório de Progresso: Implementação de I/O e Renderização Gráfica

Objetivo da Etapa: Nesta fase do projeto, a equipe concentrou-se na implementação da comunicação entre a UCP e os periféricos, utilizando a técnica de Entrada e Saída Mapeada em Memória (Memory-Mapped I/O). O foco foi estabelecer o controle visual do jogo através do Bitmap Display e a captura de comandos do jogador via teclado simulado no MARS versão 4.5.

1. Mapeamento e Atualização de Vídeo (Bitmap Display)

Para renderizar os gráficos do jogo, estamos aplicando na prática a manipulação direta de memória, tratando a tela como uma grade contínua na memória principal.

- **Definição do Endereço Base:** O Bitmap Display é configurado no MARS para monitorar um segmento de memória a partir de um endereço base no início da *Heap* (0x10040000).
- **Renderização por Coordenadas:** A resolução de 256 x 256 pixels exige a conversão de coordenadas cartesianas (X, Y) em um endereço linear de memória.
- **Cálculo de Deslocamento (Offset):** O endereço exato para desenhar um segmento da cobra é calculado via fórmula:
$$\text{Endereço Alvo} = \text{Endereço Base} + ((Y * \text{Largura da Tela}) + X) * 4.$$
- **Escrita na Tela:** Utilizamos a instrução **sw** (*Store Word*) para injetar o valor hexadecimal da cor diretamente no espaço de memória calculado,

alterando o pixel no Bitmap Display.

2. Interação com o Teclado (Técnica de Polling)

Devido às limitações do simulador MARS, a leitura assíncrona do teclado não é viável, sendo implementada a técnica de *polling*.

- **Verificação de Status:** O programa realiza checagens contínuas no Registrador de Controle do Receptor (0xFFFF0000).
- **Deteção de Evento:** Avalia-se o bit menos significativo (Bit 0). Se o valor for 1, indica que um dado de caractere está disponível.
- **Captura do Dado:** O programa lê o Registrador de Dados do Receptor (0xFFFF0004), capturando o código ASCII da tecla pressionada.
- **Atualização de Rota:** O código ASCII é traduzido em uma sequência lógica que atualiza os parâmetros de movimentação no próximo ciclo do *Game Loop*.

3. Relação Arquitetural: Processador MIPS e o Modelo IAS (Von Neumann)

A implementação estabelece um paralelo direto com o modelo IAS (Von Neumann), conforme os requisitos do TAP.

- **Programa Armazenado:** Instruções (.text) e dados (.data) compartilham o mesmo espaço lógico de endereçamento da memória principal.
- **Ciclo Fetch-Execute:** O processador opera regido pelo *Program Counter* (PC), exercendo a mesma função do *Instruction Address Register* (IAR) do modelo IAS.
- **Evolução dos Registradores:** Enquanto o IAS dependia de um único Acumulador (AC), o MIPS utiliza 32 registradores de propósito geral, permitindo manter endereço de vídeo, cores e coordenadas simultaneamente na UCP.
- **Arquitetura Load/Store:** Diferente do IAS, a relação com a memória no MIPS ocorre exclusivamente via instruções `lw` e `sw`, essencial para a

configuração de Entradas e Saídas.

4. Implementação Prática e Resultados Obtidos

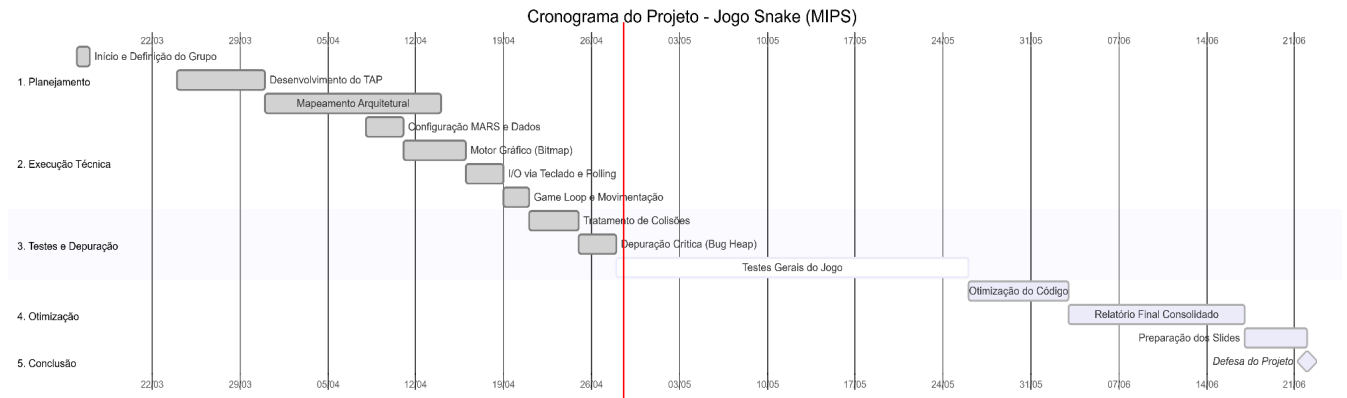
O protótipo funcional atende aos requisitos de movimentação e renderização.

- **Estruturação de Dados:** Utilizamos Arrays Paralelos (`cobra_col` e `cobra_lin`) na seção `.data` para representar o corpo da cobra.
- **Lógica de Deslocamento:** O sistema desloca os valores dos vetores de trás para frente, atualizando apenas a cabeça conforme a direção do teclado.
- **Tratamento de Colisões:** Implementamos sub-rotinas de verificação de limites de tela e autocolisão, além de um laço de varredura que impede a geração da maçã sobre o corpo da cobra.
- **Depuração Crítica:** Identificamos um erro de *Address out of range* devido ao conflito entre o segmento `.data` e o endereço base de vídeo em `0x10010000`. A solução foi isolar o Bitmap Display na área de *Heap* (`0x10040000`).

5. Gestão de Projeto (Gantt e Kanban)

A equipe utiliza um **Gráfico de Gantt** para o controle de cronograma por datas e um **Quadro Kanban** para a gestão visual de tarefas.

- **Status Atual:** As fases de planejamento, TAP e configuração inicial estão concluídas.
- **Antecipação de Prazos:** Embora o cronograma previsse o desenvolvimento da lógica para maio, a equipe já possui um protótipo funcional em abril, permitindo maior foco em testes e otimização.



6. Metodologia: Uso de Inteligência Artificial

Com o intuito de maximizar o aprendizado e adotar práticas modernas, a equipe utilizou ferramentas de Inteligência Artificial Generativa como assistentes de apoio. O uso foi focado no esclarecimento de conceitos arquiteturais de baixo nível, suporte na depuração de erros de endereçamento de memória e auxílio na estruturação formal da documentação e cronogramas. A tecnologia atuou como um acelerador produtivo, permitindo à equipe focar na compreensão das engrenagens do processador e na lógica de programação em Assembly.